

기말 프로젝트

Cardio Care 시스템 구현

2026. 06. 22.

목차

1. 개요	1
1.1. 문제 정의 및 사용 목적.....	1
2. EDA 및 전처리.....	1
2.1. EDA.....	1
2.2. EDA 결과에 근거한 전처리.....	2
3. MLflow 실험 기반 모델 비교 및 최종 결정	2
3.1. MLflow 실험 기반 모델 비교 및 최종 결정	2
4. 테스트 및 패키징.....	3
4.1. 테스트 종류.....	3
5. 드리프트(Drift) 결과 및 재학습/피드백 루프 계획.....	4
5.1. 드리프트(Drift) 결과	4
5.2. 재학습/피드백 루프 계획.....	5
6. 서빙 선택	5
6.1. 서빙: On-Device	5
7. 한계 및 윤리적 고려사항.....	6
7.1. 한계	6
7.2. 윤리적 고려사항.....	7
8. AI 사용 명시	7
8.1. 제약 사항	7
8.2. 사용 방식	7

1. 개요

1.1. 문제 정의 및 사용 목적

본 프로젝트는 CardioCare(임상 데이터로부터 심장병 발병 가능성을 예측하여 심장 전문의의 의사결정을 지원하는 시스템)를 주제로 종단간 ML 라이프사이클 과정을 구현하는 것을 목표로 한다. 심장병 진단은 환자의 생명에 직결되는 중요한 결정이다. 이때 본 시스템은 심장 전문의의 의사결정을 보조하는 도구로서, 심장병 여부를 결정하는 것이 아닌 전문의의 결정에 도움이 되도록 정보를 제공하는 것이 목적이다. CardioCare는 절대 단독으로 결정하는 시스템이 아니다.

2. EDA 및 전처리

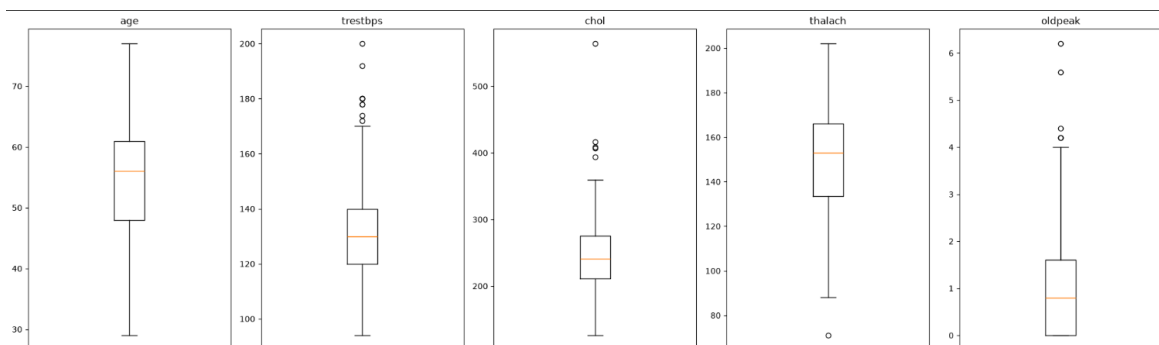
2.1. EDA

본 프로젝트에서 사용한 데이터는 고전적인 Cleveland 부분 집합 데이터셋이다. 탐색적 데이터 분석(Exploratory Data Analysis, EDA) 단계에서는 데이터셋의 특징을 파악하는 것을 목표로 한다. 이를 위해 데이터 일부 출력 확인, 타깃 클래스 분포 확인, 결측치 및 이상치 확인을 진행하였다. 다음은 타깃 클래스의 분포를 출력한 결과이다.

target	
0	0.541254
1	0.458746

[그림 1] "타깃 클래스 분포"

타깃 클래스는 0이면 정상, 1이면 심장병으로 구분되는 데이터이다. 따라서 위 결과는 데이터셋이 약 54%는 정상, 약 46%는 심장병으로 나누어져 있음을 의미한다. 정상과 심장병의 비율이 50%에 가깝게 균형 잡혀 있으며, 모델 학습 시 데이터 쏠림 현상이 발생하지 않아 정확도를 신뢰할 수 있음을 알 수 있다. 다만 심장병 진단이라는 주제의 특성상 실제 심장병 환자를 정상으로 잘못 예측(False Negative)하는 것이 매우 치명적이므로, 최종적인 모델 평가에서는 재현율(Recall)을 주된 평가 지표로 사용하는 것이 적절하다. 다음은 데이터셋의 연속형 특성들에 대한 box plot을 그린 결과이다.



[그림 2] "연속형 특성들의 box plot"

이는 각 특성의 IQR을 기준으로 이상치를 눈에 띄게 표현한 것이다. 'trestbps', 'chol', 'thalach', 'oldpeak' 특성에 비정상적으로 크거나 작은 값이 다수 존재했다. 또한 열별, 전체 결측치 개수 확인 결과 'ca', 'thal' 특성에 결측치가 존재하였다. 이러한 데이터셋의 특징들을 종합하면 전처리 과정에서 무엇을 해야 하는지 알 수 있다.

2.2. EDA 결과에 근거한 전처리

EDA 결과를 종합하면 데이터셋의 타깃 클래스 분포는 균형 잡혀 있지만, 결측치와 이상치가 다수 포함되어 있는 상태이다. 이러한 값들은 모델의 정확한 예측에 악영향을 주므로, 적절히 처리되어야 한다. 또한 열 전체가 비어 있거나 같은 값만 존재하는 경우 모델의 예측에 아무런 도움이 되지 않지만 복잡도는 증가시키므로 제거해야 하며, 모든 열에 대해 같은 값을 갖는 행들은 과적합을 유발할 수 있어 하나만 남기고 제거해야 한다. 즉, 빈 컬럼과 중복 데이터는 단순히 제거하는 것으로 해결하였다. 이때 데이터를 제거하는 것은 데이터셋의 형태를 바꾸는 작업이므로, Train/Test 셋을 분할하기 이전에 데이터에 먼저 적용한다.

본 프로젝트는 이러한 전처리를 '이상치 클램핑(Clamping)', '결측치 KNN 대체', '빈 컬럼 및 중복 데이터 제거' 3개로 구분하였다. 이상치 클램핑은 IQR 범위를 벗어나는 것을 기준으로 탐지한 이상치를 해당 범위 내의 최소/최대치로 제한하는 것이다. 심장병 여부를 진단할 때, 전체적인 값의 분포에서 크게 벗어난 수치(이상치)는 큰 힌트가 될 수 있는 값이므로 단순히 제거해서는 안 된다. 따라서 허용되는 범위에서 가장 큰 값 또는 작은 값으로 변경함으로써 모델이 예측에 활용할 수 있도록 하되, 전체 데이터에 과도한 왜곡을 일으키는 것을 방지하였다.

EDA를 진행한 데이터셋에서 결측치는 총 6개로 소량 존재하였다. 단순히 결측치가 있는 행을 제거해도 당장의 문제는 없으나, 다른 데이터셋에서도 결측치가 적을 것이라고 확신할 수 없으므로 단순 제거는 배제하였다. 결측치를 해당 특성의 평균 또는 중앙값으로 대체하는 것은 해당 열에 대해서만 계산된 값을 적용하는 것이므로 다른 특성들과의 상관관계가 무시되는 문제가 존재한다. 또한 결측치가 많은 경우 모든 결측치가 평균 또는 중앙값으로 대체되어 데이터의 분포에 인위적인 영향을 주므로, 데이터의 기존 분산이 오염되어 모델의 학습에 악영향을 줄 수 있다. 따라서 KNN 대체를 통해 다른 특성들과의 상관관계를 반영한 맞춤 데이터로 결측치를 대체함으로써, 데이터의 통계적 특성을 유지하면서 결측치를 해결한다.

이러한 전처리 파이프라인은 새 데이터에 그대로 재적용 가능한 함수 또는 'sklearn.pipeline.Pipeline' 형태로 구현하였다. 이상치 클램핑은 'IQRClamp' 함수로, 빈 컬럼 및 중복 데이터 제거는 'clean_data' 함수로 구현되었으며, KNN 대체의 경우 'KNNImputer'를 Pipeline의 첫 절차로 삽입하여 구현하였다.

3. MLflow 실험 기반 모델 비교 및 최종 결정

3.1. MLflow 실험 기반 모델 비교 및 최종 결정

MLflow 실험은 'train.py' 내에서 진행되었다. 'Logistic Regression', 'SVC', 'Random Forest' 3가지의 모델을 모두 같은 데이터셋으로 학습 후 예측을 수행하여 결과를 비교하였다. 모든 모델은 5-fold 교차 검증을 거쳤으며, 그 과정에서 사용된 평가 지표, 생성된 아티팩트(모델 파일)는 모두 MLflow를 통해 결정론적으로 추적 및 기록하여 실험

의 재현성을 확보하였다.

심장병 진단을 보조하는 본 시스템의 목적상 심장병 환자를 정상으로 오진하는 경우(False Negative) 환자의 생명에 치명적인 위험을 초래할 수 있다. 반면 정상을 심장병으로 오진하는 경우(False Positive)는 즉각적인 생명의 위협으로 직결되지는 않아 비교적 중요도가 낮다. 따라서 최종 모델을 선택할 때는 정확도(Accuracy)보다 실제 환자를 최대한 많이 찾아내는 재현율(Recall) 지표를 최우선 평가 지표로 삼는다. 모델 간의 성능 비교에는 테스트 데이터 누수를 차단해야 한다. 이를 위해 훈련 데이터(Train Set) 내에서 5-fold교차 검증을 수행하였으며, 특정 데이터 분할에 편향되지 않은 지표인 CV 평균 재현율(Mean CV Recall)을 기준으로 모델의 성능을 평가하였다. 다음은 해당 지표를 포함한 모델 비교표와 모델 별 혼동 행렬(Confusion Matrix)이다.

모델	CV 평균 Recall	Balanced Accuracy	Precision	Recall	F1
Logistic Regression	0.7751	0.7841	0.7778	0.75	0.7636
SVC	0.7755	0.7689	0.75	0.75	0.75
Random Forest	0.7395	0.7511	0.7407	0.7143	0.7273

```
Confusion Matrix:
[[27  6]
 [ 7 21]]
```

[그림 3] "Logistic Regression"

```
Confusion Matrix:
[[26  7]
 [ 7 21]]
```

[그림 4] "SVC"

```
Confusion Matrix:
[[26  7]
 [ 8 20]]
```

[그림 5] "Random Forest"

대체로 'Logistic Regression'과 'SVC'가 우수한 모습을 보였고, 'Logistic Regression'이 절대적인 정답 수가 미세하게 더 많아 전반적인 수치는 근소하게 우수하다. 하지만 본 시스템의 최우선 평가 지표인 CV 평균 재현율에서는 SVC가 더 높은 수치를 기록하였으므로 최종 유력 모델로 선정하였다. 추가로 유력 모델에 대해서는 하이퍼 파라미터 튜닝을 수행하도록 코드를 작성하였다.

4. 테스트 및 패키징

4.1. 테스트 종류

본 프로젝트에서는 시스템의 신뢰성과 무결성을 보장하기 위해 다음과 같은 4가지 단위 테스트(Unittest)를 수행하였다.

1. 입출력 형태 일치 검증

모델에 N명의 환자 데이터를 입력했을 때, 정확히 N개의 예측 결과가 반환되는지 테스트하였다. 전처리 파이프라인 또는 결측치 대체 과정에서 특정 행이 임의로 삭제되는 등의 데이터 변형이 발생하지 않고 파이

프라인이 구조적으로 안정적인지 확인하기 위함이다.

2. 예측 확률 유효성 검증

모델이 반환하는 예측 확률이 항상 $[0, 1]$ 범위 내에 있고, 각 클래스 확률의 합이 약 1인지 테스트하였다. 모델 내부의 스케일링이나 특정 연산 과정에서 비정상적인 값이 발생하지 않음을 증명하고, 신뢰할 수 있는 정확한 확률값을 제공하기 위함이다.

3. 임상적 입력값 범위 검증

환자의 콜레스테롤(chol)과 같은 특정 수치가 인간의 생물학적 한계(예: 0~600)를 벗어나는 극단적인 값으로 입력되었을 때, 전처리를 통해 올바르게 처리하는지 테스트하였다. 오기입이나 기기 오류로 인해 극단적인 이상치가 입력되면 데이터 왜곡으로 인해 예측 성능이 저하될 수 있다. 따라서 파이프라인이 비정상적인 수치를 모델 학습 및 추론 이전에 의도한 범위 내로 제한하여 사용하고 있는지 검증하였다.

4. 파이프라인 결정론적 동작 검증

난수 시드(Seed)를 고정한 상태에서 동일한 환자의 데이터를 여러 번 입력했을 때 동일한 예측 결과가 출력되는지 테스트하였다. 같은 데이터를 입력해도 무작위성에 의해 진단 결과가 매번 바뀌면 시스템을 신뢰할 수 없기 때문이다.

5. 드리프트(Drift) 결과 및 재학습/피드백 루프 계획

5.1. 드리프트(Drift) 결과

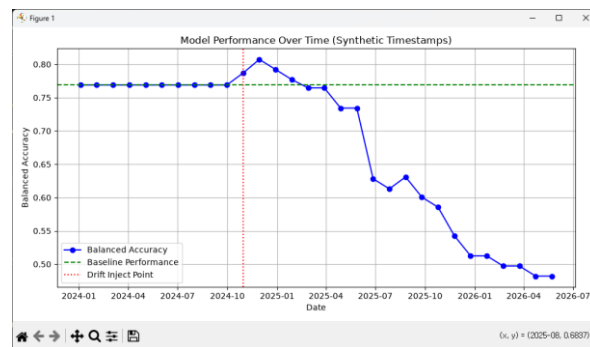
본 시스템에서는 추론 발생 시 타임스탬프, 모델 버전, 입력 shape, 예측값, 실제 정답을 'inference.log' 파일로 남기는 로깅을 수행하였다. 이후 드리프트 감지가 정상적으로 작동하는지 확인하기 위해, 테스트 데이터의 연속형 특성 중 'age', 'chol', 'thalach', 'oldpeak'에 대해 평균 및 분산을 이동시키는 시뮬레이션을 진행하였다. 해당 특성들에 대해, 원본 학습 데이터 분포와 시뮬레이션 데이터 분포 간의 KS 검정에서 p-value가 0.05 미만으로 떨어졌고 정상적으로 드리프트가 탐지되었다. 드리프트가 탐지된 경우 플래그를 설정하도록 코드를 구현하였다. 다음은 그 실행 결과이며, 유일하게 어떤 조작도 가하지 않은 'trestbps' 특성은 정상적인 상태임을 알 수 있다.

```
[Data Drift K-S Test 결과]
Feature: age      | p-value: 0.0219 | [DRIFT DETECTED]
Feature: trestbps | p-value: 0.7668 | [OK]
Feature: chol     | p-value: 0.0000 | [DRIFT DETECTED]
Feature: thalach  | p-value: 0.0000 | [DRIFT DETECTED]
Feature: oldpeak  | p-value: 0.0000 | [DRIFT DETECTED]

[모델 성능 변화 (Balanced Accuracy)]
원본 테스트셋 성능 : 0.7689
드리프트 테스트셋 성능 : 0.6131
성능 하락 폭      : 0.1558
```

[그림 6] "드리프트 결과 터미널 출력"

이후 시간에 따른 지표 변화를 시각화하기 위해 'Balanced accuracy' 지표를 기준으로 합성 타임스탬프 기반의 시계열 그래프를 생성하였다.



[그림 7] "시간에 따른 지표 변화 (Balanced Accuracy 기준)"

총 30개월의 기간을 가정하였으며, 시간의 흐름에 따라 데이터 드리프트가 점진적으로 심해지도록 설계하였다. 그 결과 데이터 드리프트가 발생하기 시작한 시점부터 지표가 점진적으로 하락하는 것을 확인할 수 있다. 극초기에는 오히려 지표가 상승하기도 하는데, 이는 아슬아슬하게 정답을 맞지 못했던 예측 사례 일부가 미세한 수치 조절에 의해 우연히 정답을 맞게 된 것으로 추측된다. 하지만 얼마 안가 데이터 드리프트가 심해지며 지표가 급락한다.

5.2. 재학습/피드백 루프 계획

드리프트는 최신 데이터로 처음부터 다시 학습하는 '재학습(Retrain)' 또는 '파라미터 재조정(Fine-tuning)'을 통해 개선할 수 있다. 기본적으로 6개월 주기의 정기적인 재학습을 수행하되, 모니터링을 통해 KS 검정 기반의 드리프트 경고가 발생하거나 성능 지표가 임계치 이하로 하락하는 경우 즉시 재학습을 수행하도록 설계한다. 또한 모델의 예측 성능을 모니터링하고 재학습 데이터를 확보하기 위해, 전문의가 검증(Human-in-the-loop)한 결과에 기반한 피드백 루프를 구축한다. 단순히 모델의 예측값을 그대로 재학습 데이터로 활용할 수는 없다. 전문의의 최종 확진 결과가 반영된 데이터를 실제 정답으로 삼아 재학습 데이터로 사용하여 피드백 루프를 구성해야 모델이 정확한 데이터를 기반으로 학습을 수행할 수 있다.

실시간으로 수집된 데이터만으로 시스템이 자동 재학습을 반복(피드백 루프)할 경우, '폭주하는 피드백 루프'에 빠질 위험성이 존재한다. 만약 일시적으로 비정상적인 데이터가 대량 유입되면, 시스템은 해당 입력에 대한 결과가 반영된 데이터로 학습을 수행하여 오염될 수 있다. 또한 모델이 예측한 결과를 정답으로 삼아 학습에 재사용하면, 모델의 편향이 심해질 가능성이 크다. 피드백 루프 계획에서 전문의가 검증한 결과를 재학습 데이터로 사용한 것은 이러한 폭주하는 피드백 루프를 막기 위함이다.

6. 서빙 선택

6.1. 서빙: On-Device

결론적으로 본 시스템은 'Model-as-a-Service'가 아닌 'On-Device' 기반의 서빙을 채택한다. 그 이유를 다음과 같이 세 가지 관점에서 설명하고자 한다.

1. 지연 시간

본 시스템은 환자의 생체 수치와 진단 결과로 이루어진 가벼운 정형 데이터를 입력 받아 예측 결과를 출력하는 가벼운 모델이다. 따라서 비교적 가벼운 하드웨어에서도 지나친 연산 시간 없이 사용이 가능할 것이다. 또한 심장병 전문가가 배치되는 수준의 대형 병원 환경에서는 성능이 더 높은 하드웨어도 충분히 지원할 수 있으리라 생각한다. 지연 시간 관점에서 On-Device가 갖는 약점은 없으며, 오히려 네트워크 상태에 영향을 받지 않아 안정적인 운용이 가능하여 훨씬 유리하다.

2. 개인정보(PHI)

개인정보 보호 관점에서는 On-Device가 압도적으로 유리하다. 환자의 생체 수치와 진단 결과 등의 정보는 유출 시 매우 치명적인 민감 정보이다. MaaS를 사용한 경우 API를 통해 데이터가 외부 서버로 전송되어야 하는데, 이는 전송 과정에서의 패킷 탈취 위험성을 내포하며 해킹에 따른 대규모 유출 리스크를 동반한다. On-Device는 모든 연산이 기기 내부에서 처리되고 원본 데이터가 외부 네트워크에 노출되지 않으므로, 데이터 보안에 훨씬 유리하다. 개인정보 보호는 법적으로도 매우 중요한 사안이므로, On-Device 방식을 채택하는데 큰 영향을 주는 관점이다.

3. 업데이트 주기

일반적인 IT 서비스와 달리, 의료 시스템에서는 쉽고 빠른 업데이트보다 안정적인 업데이트가 더 중요하다. MaaS의 경우 서버의 모델만 업데이트하면 해당 API를 사용하는 모든 기기의 업데이트가 이루어져 신속성 면에서 우위에 있다. 반대로 On-Device는 각 기기에서 따로 업데이트를 진행하므로 업데이트 주기는 길어질 수밖에 없다. 하지만 심장병 진단이라는 도메인상 해당 모델은 잦은 업데이트가 필요하지 않으며, 새로운 버전에 대한 철저한 검증 요구를 지키기 용이하다. 따라서 업데이트 주기가 긴 On-Device의 단점이 크게 작용하지 않는다.

7. 한계 및 윤리적 고려사항

7.1. 한계

본 프로젝트의 한계는 데이터의 규모가 작은 점을 들 수 있다. 사용된 데이터셋은 Cleveland 부분 집합이므로, 특정 인구통계에 국한되어 있고 그 표본의 수도 303행으로 적은 편이다. 따라서 현재 모델을 다양한 환경의 환자들에게 일반화하여 적용하기는 어렵다. 또한 현재 시스템은 파이프라인 설계가 전반적으로 연속형 특성에 치중되어 있다. 'cp' 또는 'restecg' 등의 범주형 데이터들은 크기나 순서가 없는 데이터임에도 원본 숫자 그대로 학습에 투입되어 마치 숫자의 크기가 의미를 갖는 것처럼 학습되었을 가능성이 크다.

따라서 1주의 시간이 더 주어진다면, 우선 더 다양한 인구통계를 반영한 추가 데이터셋을 확보하여 학습의 품질을 더 높이고 연속형 특성 이외의 다른 특성들에 대한 처리를 더 개선하겠다. 예를 들어, 범주형 데이터의 경우 원

핫 인코딩(One-Hot Encoding)을 적용하는 등의 개선이 분명히 이루어질 수 있으리라 생각한다. 추가로 데이터 모니터링에서도 현재는 연속형 특성들만 탐지하고 있어 다른 특성에 대한 탐지법을 찾고 적용할 필요가 있다.

7.2. 윤리적 고려사항

심장병 진단은 그 결과에 따라 환자의 생명이 갈리는 매우 민감한 주제이다. 본 심장병 발병 가능성 예측 시스템은 심장 전문의의 의사결정을 보조하는 도구이며, 시스템이 단독으로 최종 결정을 할 수 없다. 따라서 시스템의 예측을 맹신하는 것이 아닌 전문의의 최종적인 결정을 따라야 한다.

8. AI 사용 명시

8.1. 제약 사항

본 프로젝트의 진행 과정에서 생성형 AI(Copilot, ChatGPT, Claude 등)는 보일러플레이트 코드 작성과 디버깅에 한하여 사용한다. 실제로 사용한 AI는 Gemini 3 Pro standard이다.

8.2. 사용 방식

본 프로젝트는 과제 설명 문서에서 명시한 필수 과업 순서를 따라 진행했으며, 그 과정에서 보일러플레이트 코드 작성에 필요한 함수명과 그 사용법을 묻고 작성한 코드의 평가를 요청하는 정도의 활용이 주로 이루어졌다. 주요 코드 작성과 의사결정은 강의자료(DataCamp 강의 동영상 및 ppt)에 포함되어 있는 이론적 내용과 실습 코드의 형태를 직접 참고하며 진행하였다. 특정 모델이 사용하는 하이퍼 파라미터의 종류와 같이 강의 내용에서 명확히 명시되지 않은 정보를 얻는 데는 AI를 활용하였다. 가장 직접적으로 코드 생성을 활용한 부분은 'inference.py'의 작성으로, 'train.py'의 실행 결과로 생성되는 MLflow의 메트릭을 기반으로 최적의 모델을 동적으로 탐색('mlruns/' 내부 파일 탐색)하는 코드의 구현에 도움을 받았다. Github Actions CI 워크플로인 'ci.yml'의 작성은 AI의 코드 생성으로 진행되었다. 'monitor.py'를 작성할 때는 'logging' 라이브러리의 사용법과 터미널 출력의 형태, MLflow 파일을 불러오는 방법, pyplot 그래프 구성에 대해 질문하여 답을 얻었다.